Documentación técnica del pipeline

Modelo predictivo del Índice de Vulnerabilidad y Carencias (IVC) — Bogotá

Este documento describe, paso a paso, el proceso ejecutado para construir, validar y preparar para producción un modelo de regresión que predice el índice **ivc_n** a partir de variables socioeconómicas, catastrales y demográficas de Bogotá. El flujo combina un proceso de ETL en AWS Glue, modelado con PyCaret/scikit-learn en Amazon SageMaker (JupyterLab), y persistencia del modelo final en Amazon S3.

Etapa 1 · Extracción y transformación de datos (ETL)

Los datos crudos se leyeron desde un bucket de Amazon S3 (`s3://databogota/bogota_ivc_thesis_ready.csv`) usando la librería `awswrangler`. El dataset original contenía 62 columnas.

1.1 Eliminación de columnas con fuga de datos (data leakage)

Durante la primera comparación de modelos se detectaron métricas irrealmente perfectas ($R^2 \approx 1.0000$, MAE/MSE ≈ 0), señal característica de que el target estaba filtrado en las propias variables predictoras. Se identificaron y eliminaron tres columnas derivadas directamente del índice objetivo:

- `ivc_formula`
- `categoria_ivc`
- `perfil_vulnerabilidad`

```
df = df.drop(columns=['ivc_formula', 'categoria_ivc', 'perfil_vulnerabilidad'])
```

Resultado: el dataset quedó en 58 columnas útiles, sin fuga de información hacia el target.

Etapa 2 · Configuración del entorno de modelado

El entrenamiento se realizó en un kernel dedicado de JupyterLab (SageMaker Studio) con **Python 3.10**, creado específicamente porque PyCaret no es compatible con Python 3.12 (la versión por defecto del entorno). Se instalaron PyCaret, ipykernel y awswrangler en un entorno conda aislado (`pycaret_env`).

Con la data ya sin fuga de información, se inicializó el experimento de PyCaret usando **todas las columnas restantes** como candidatas a feature — la selección de variables relevantes se realiza después, con base en la importancia que arroje el modelo ganador, nunca antes de conocerlo.

```
experiment = setup(data=df, target='ivc_n', session_id=123)
```

PyCaret aplica automáticamente imputación de nulos, codificación de variables categóricas y escalado numérico dentro de este paso.

Etapa 3 · Comparación y selección del modelo

Se ejecutó `compare_models(sort='R2')`, que entrena y evalúa múltiples algoritmos de regresión con validación cruzada. Los resultados principales fueron:

Modelo	MAE	MSE	RMSE	R2	TT (seg)
LightGBM (ganador)	0.0159	0.0007	0.0260	0.8856	104.39
Extra Trees Regressor	0.0162	0.0007	0.0267	0.8795	7.22
Random Forest Regressor	0.0161	0.0007	0.0267	0.8794	14.74
Gradient Boosting Regressor	0.0187	0.0008	0.0291	0.8566	4.15
Decision Tree Regressor	0.0234	0.0015	0.0390	0.7423	0.37
AdaBoost Regressor	0.0376	0.0022	0.0467	0.6305	1.75
K Neighbors Regressor	0.0315	0.0023	0.0477	0.6156	0.24

Modelo ganador: LightGBM (LGBMRegressor), con el mejor R^2 (0.8856) del grupo. Los modelos basados en árboles/ensambles (Extra Trees, Random Forest, Gradient Boosting) obtuvieron desempeños cercanos, lo que refuerza la validez de las variables como predictoras genuinas del índice. Como el modelo ganador ya era conocido, en las siguientes fases se usó `create_model('lightgbm')` en lugar de repetir la comparación completa.

Etapa 4 · Validación robusta del modelo

Sobre el estimador puro extraído de PyCaret (`LGBMRegressor`) y los datos ya transformados (`X, y = get_config('X'), get_config('y')`, con forma final (43 408, 58) / (43 408,)), se aplicaron tres esquemas de validación complementarios en scikit-learn puro.

4.1 Validación cruzada de 10 folds (PyCaret)

Métrica	Media	Desv. estándar
MAE	0.0159	0.0003
MSE	0.0007	0.0001
RMSE	0.0260	0.0010
R2	0.8856	0.0078

4.2 Particiones train/test (70/30, 80/20, 90/10)

Split	N train	N test	R2	MAE	RMSE
70/30	30 385	13 023	0.8891	0.0161	0.0258
80/20	34 726	8 682	0.8901	0.0160	0.0257
90/10	39 067	4 341	0.8991	0.0156	0.0247

4.3 Validación cruzada K-Fold (5 y 10 folds, scikit-learn)

K folds	R2 medio	R2 std	MAE medio	RMSE medio
5	0.8880	0.0036	0.0158	0.0258
10	0.8893	0.0063	0.0157	0.0257

Los tres esquemas convergen en un R^2 entre 0.885 y 0.899, lo que confirma que el desempeño del modelo es estable y no depende de una partición particular de los datos.

Etapa 5 · Encapsulado en pipeline de scikit-learn y entrenamiento final

El estimador se encapsuló en un `sklearn.pipeline.Pipeline` formal, y se entrenó de forma definitiva sobre la partición 80/20:

```
pipeline_final = Pipeline([('regressor', clone(best_model))])
pipeline_final.fit(X_train, y_train)
```

Métrica	Valor
R2 final	0.8901
MAE final	0.0160
RMSE final	0.0257

Etapa 6 · Persistencia y almacenamiento en S3

El pipeline entrenado se serializó con `joblib` y se subió a Amazon S3 para quedar versionado y disponible fuera del notebook:

```
joblib.dump(pipeline_final, 'pipeline_final_tesis.pkl')
wr.s3.upload(local_file='pipeline_final_tesis.pkl',
    path='s3://databogota/modelos/pipeline_final_tesis.pkl')
```

Resultado: modelo disponible en `s3://databogota/modelos/pipeline_final_tesis.pkl`

Etapa 7 · Próximos pasos: despliegue en producción

Con el modelo ya persistido en S3, las actividades pendientes para completar el pipeline de producción son:

- Extraer la importancia de variables del modelo LightGBM y, si se decide reducir el conjunto de features, repetir el setup solo con las variables seleccionadas.
- Empaquetar el modelo en formato `model.tar.gz`, requerido por SageMaker.
- Escribir el script de inferencia (`inference.py`) con las funciones `model_fn, input_fn, predict_fn, output_fn`.
- Desplegar un endpoint de SageMaker en tiempo real con `SKLearnModel.deploy()`.
- Probar el endpoint con datos externos (fuera de Bogotá) para validar la compatibilidad de columnas.

Documento generado automáticamente a partir de la ejecución del notebook `Modelo_bogota.ipynb` en Amazon SageMaker Studio (JupyterLab).